

added for convenience and are not part of the code.

```
1. import numpy as np
2. import scipy as sp
3. A=np.array([[11,22,33],[44,55,66],[
77,88,888]])
4. P, L, U = sp.linalg.lu(A)
5. print(P)
6. print(L)
7. print(U)
8. print(P@L@U)
```

Figure 5 shows the output of the code. Lines 1 and 2 import packages NumPy and SciPy. A matrix A is created in line 3. Please keep in mind that we use matrix A throughout this section. Line 4 factorizes matrix A into three matrices — P, L, and U. Lines 5 to 7 print the matrices P, L, and U. From Figure 5, it is clear that P is a permutation matrix, L is a lower triangular matrix and U is an upper triangular matrix. Finally, line 10 multiplies the three matrices and prints the product matrix. Again, from Figure 5 it is clear that the product matrix $P@L@U$ equals the original matrix A. Hence, the decomposition (factorization) property holds true. Further, you can verify from Figure 6 that matrices L, U, and P are more sparse (a lot of zero elements) than matrix A.

Now let us discuss eigendecomposition, in which a square matrix is represented in terms of its eigenvalues and eigenvectors. Though it is easy to calculate eigenvalues and eigenvectors using Python, a theoretical explanation about both is beyond the scope of our discussion. However, I encourage you to learn about these concepts (if you don't know what they are already) so that you will have a clear idea about the actual operations you are performing. Again, the Wikipedia article on eigenvalues and eigenvectors is a good starting point. Now consider Figure 6, which shows the code for eigendecomposition.

In Figure 6, line 1 finds the eigenvalues and eigenvectors. Lines 2

```
[[0. 1. 0.]
 [0. 0. 1.]
 [1. 0. 0.]]
[[1.         0.         0.         ]
 [0.14285714 1.         0.         ]
 [0.57142857 0.5        1.         ]]
[[ 77.         88.         888.        ]
 [   0.         9.42857143 -93.85714286]
 [   0.         0.         -394.5        ]]
```

Figure 5: Output of LUP decomposition with SciPy

```
Lambda, Q = sp.linalg.eig(A)
print(Lambda.T)
print(Q)
print(Q@np.diag(Lambda)@sp.linalg.inv(Q))
```

```
[898.08679874+0.j -5.21687442+0.j 61.13007568+0.j]
```

```
[[-0.03903698 -0.81407387 -0.32545916]
 [-0.08001023  0.58061638 -0.93657923]
 [-0.99602936  0.01297495  0.12998336]]
```

```
[ 11.+0.j  22.+0.j  33.+0.j]
 [ 44.+0.j  55.+0.j  66.+0.j]
 [ 77.+0.j  88.+0.j 888.+0.j]]
```

Figure 6: Eigendecomposition with SciPy

and 3 print them. Notice that a similar result can also be obtained with NumPy using the line of code `Lambda, Q = np.linalg.eig(A)`. This also tells us that there is some overlap between NumPy and SciPy functions. Line 4 reconstructs the original matrix A. The code snippet `np.diag(Lambda)` in line 4 converts the eigenvalues into a diagonal matrix (we call it Λ). A diagonal matrix is a matrix in which all the elements outside the main diagonal are zero. The code snippet `sp.linalg.inv(Q)` in line 4 finds the inverse of Q (we call it Q^{-1}). Finally, the three matrices Q, Λ , and Q^{-1} are multiplied together to obtain the original matrix A. Thus, in eigendecomposition $A=Q\Lambda Q^{-1}$.

Figure 6 also shows the output of the code executed. The eigenvalues are marked with a red rectangle, the eigenvectors are marked with a

green rectangle, and the reconstructed original matrix A is marked with a blue rectangle. But what are numbers like `11.+0.j` doing in the output? Well, `11.+0.j` is the complex number representation of the integer 11.

Let us now move on to singular value decomposition (SVD). It is a generalisation of eigendecomposition. Figure 7 shows the code and output of SVD. Line 1 decomposes matrix A into three matrices U, S, and V. In line 2, the code snippet '`np.diag(S)`' converts S to a diagonal matrix. Finally, the three matrices are multiplied together to reconstruct the original matrix A. The advantage of SVD is that it can even diagonalize non-square matrices. However, the code for singular value decomposition of a non-square matrix is slightly more complicated and we won't